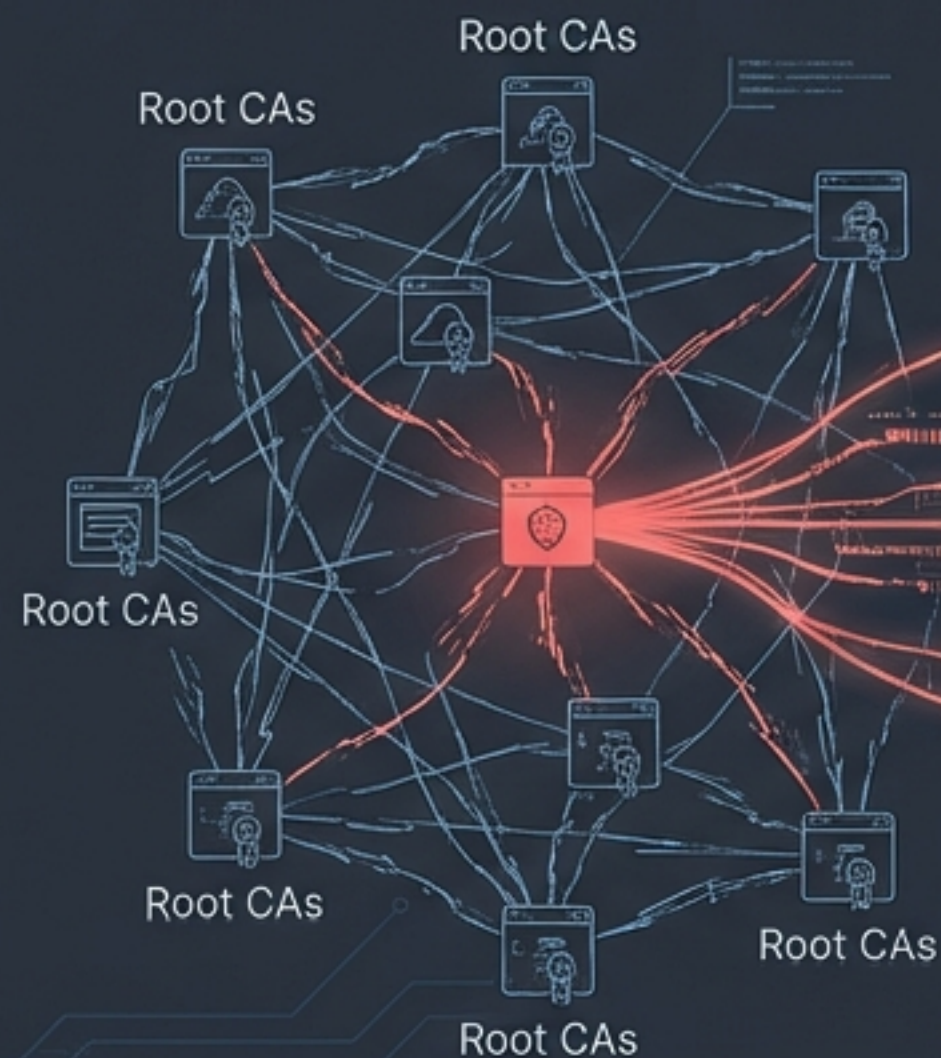


SCALE 14

Executive Summary: Moving the Trust Anchor

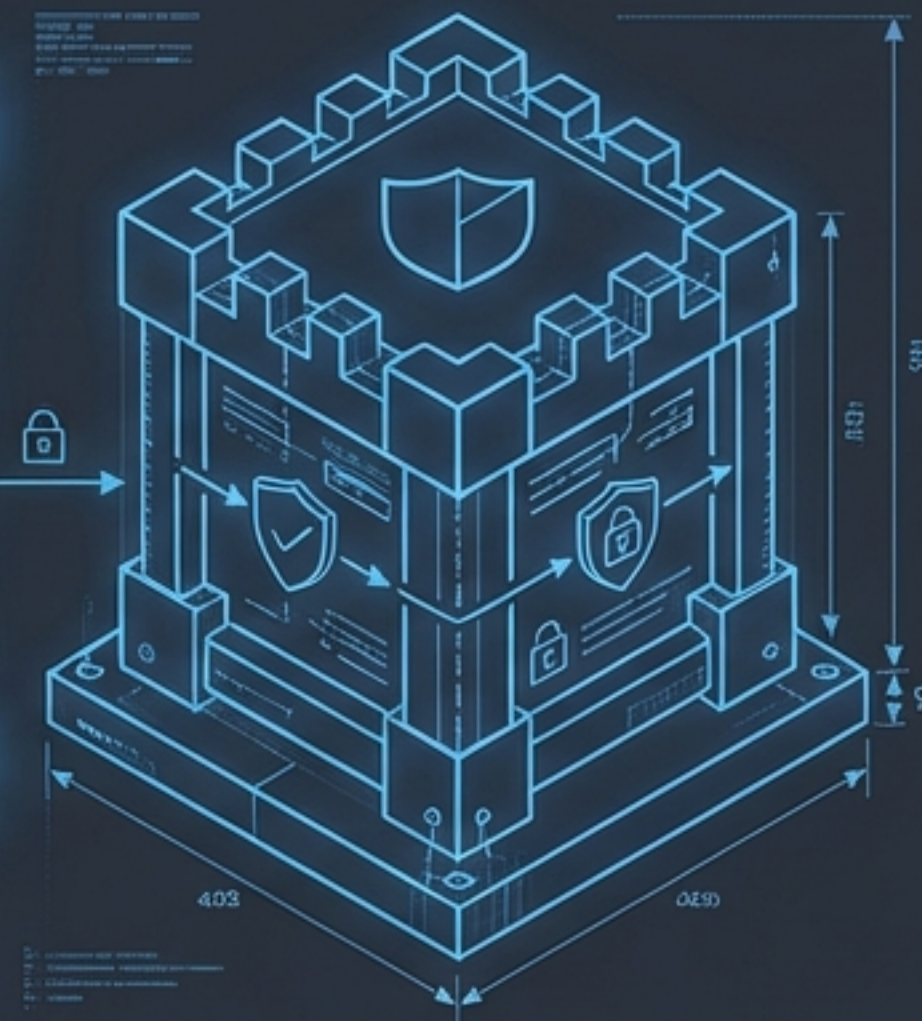
The Problem



Network Reliance (Current)

The Solution

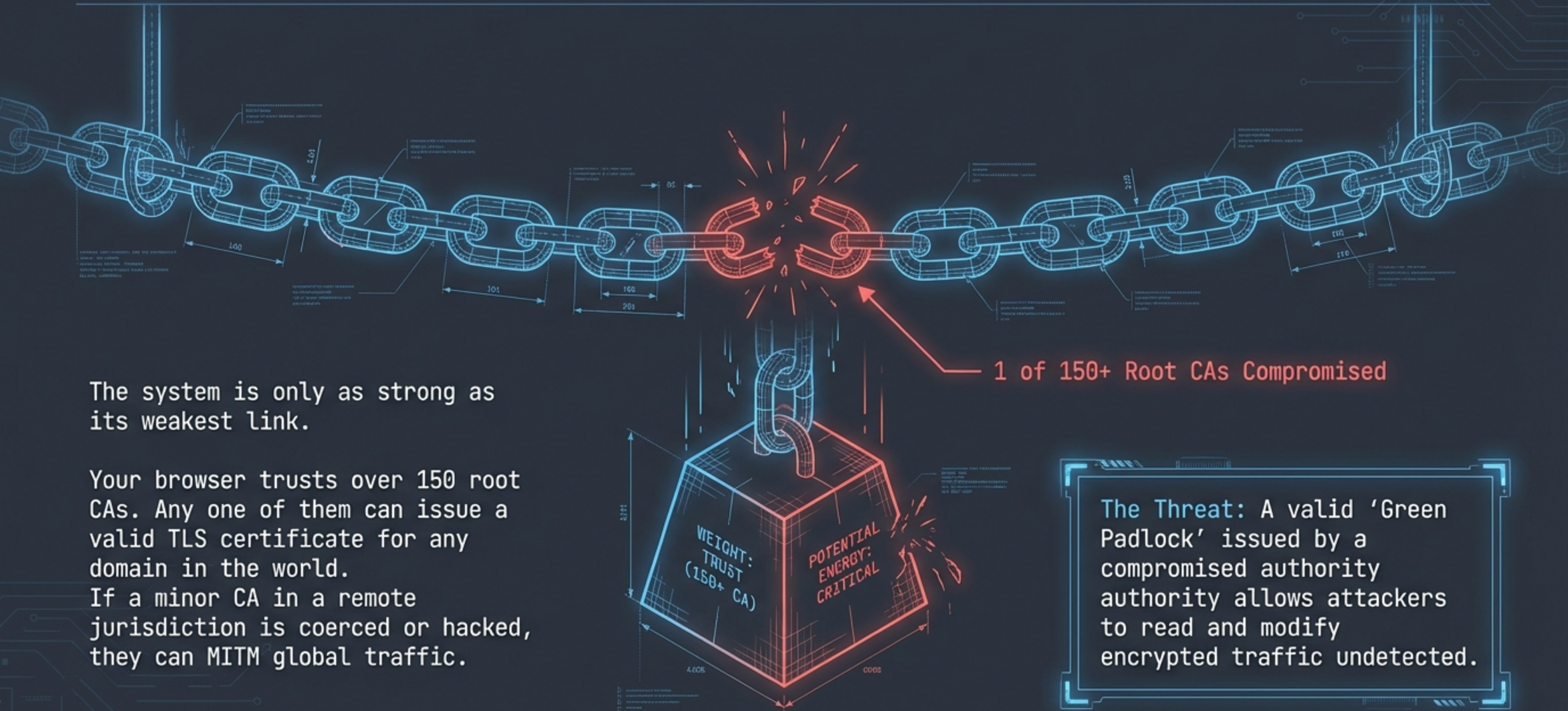
FROM (Current Model):	TO (New Model):
Trust depends on 150+ Root CAs	Trust depends on Local Service Worker
Network attackers can spoof valid certs	Code and Content are cryptographically pinned
Reactive protection (CT Logs)	Preventative protection (3-Layer Defense)



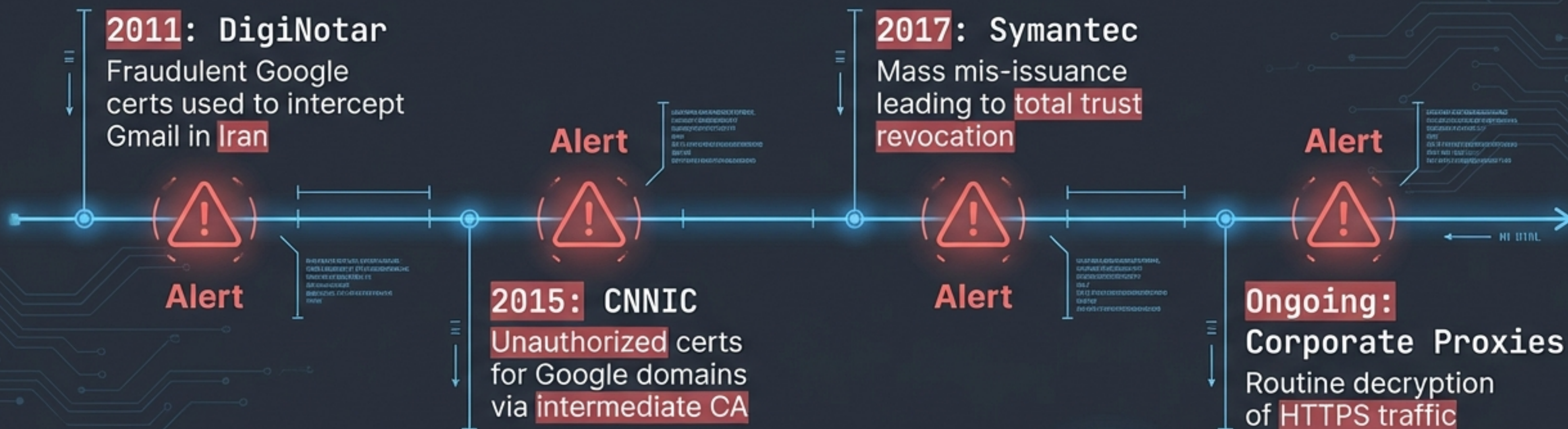
Client Verification (Proposed)

Core Architecture: 1. Code Integrity // 2. Content Authenticity // 3. Source Restriction

The CA Trust Model is Structurally Broken



Evidence of Failure: This Is Not Theoretical



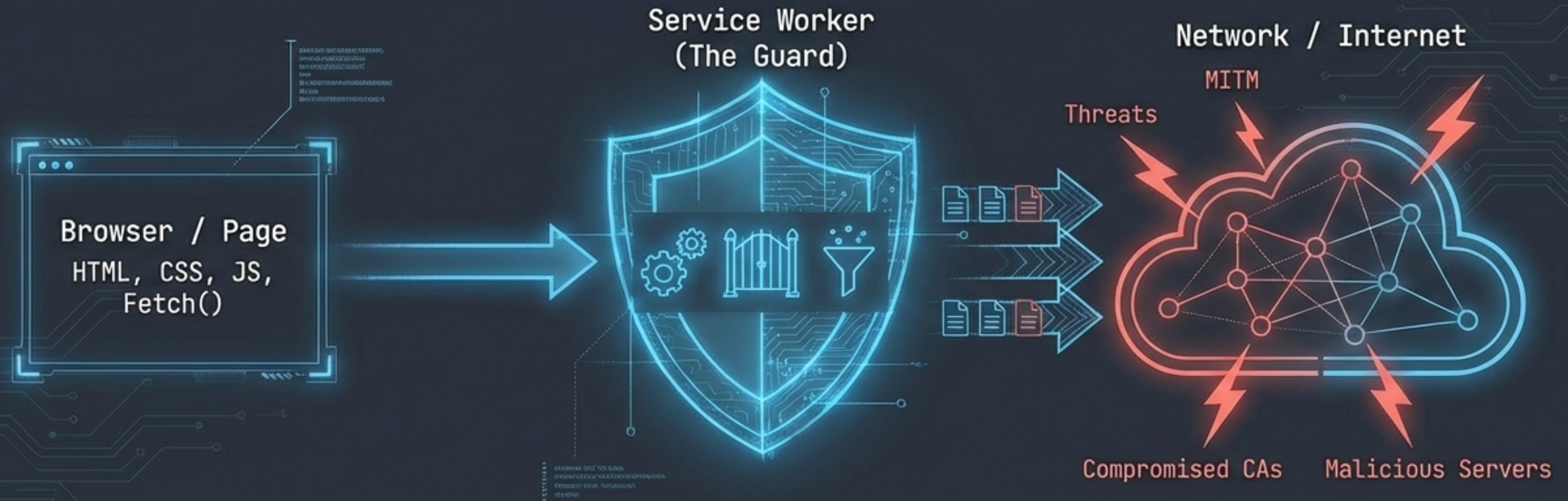
Certificate Transparency (CT) is reactive. It detects fraud after issuance. We need a preventative architecture.



Reactive Detection

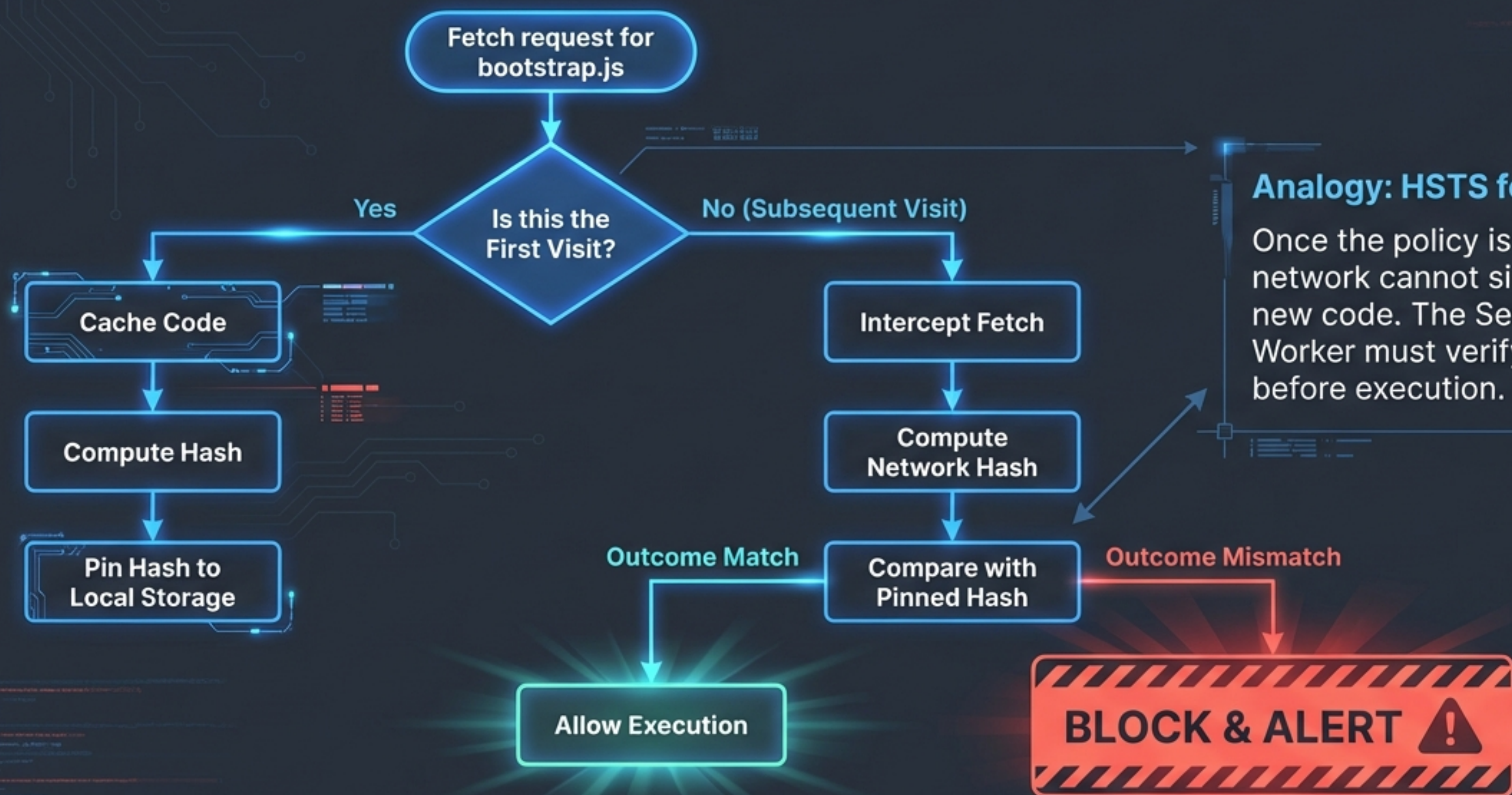


The Solution: Service Worker as Local Trust Anchor



- **Concept:** Move the 'Trust Anchor' from the Network (CAs) to the Client.
- **Mechanism:** Trust On First Use (TOFU).
- **Execution:** The SW runs before page code, acting as an independent gatekeeper.

Layer 1: Code Integrity via Hash Pinning



Analogy: HSTS for Code

Once the policy is set, the network cannot simply serve new code. The Service Worker must verify integrity before execution.

Layer 2: Content Authenticity via Signed Responses

Server
(Private Key (Ed25519))



Server
(Private Key (Ed25519))



Transmission



Service Worker
(Public Key Scanner)



Service Worker
(Public Key Scanner)

The Defense mechanism: Even if TLS is broken by a compromised CA, the attacker cannot sign the data.

1. Server signs response with Private Key.
2. Service Worker verifies with Pinned Public Key.
3. Invalid signature = **Rejected connection**.



Layer 3: Source Restriction (The Trust Registry)

ALLOWED SOURCE

trusted-cdn.com

UNKNOWN SOURCE

unknown-script.net

COMPROMISED CDN

malicious-content.com

ALLOWED
SOURCE

trusted-cdn.com

SERVICE WORKER

The "Data Room" Concept:

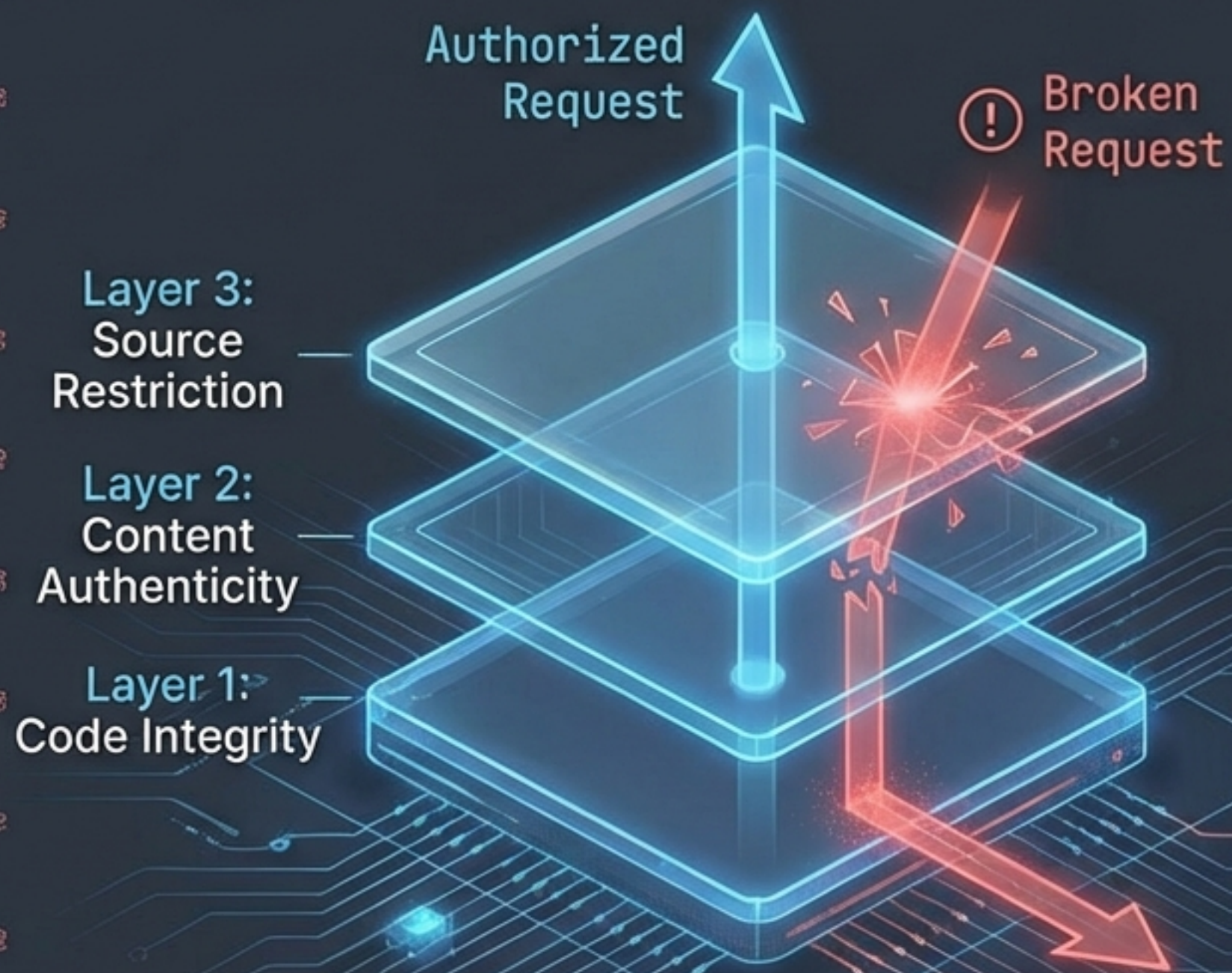
If it's not on the list,
it doesn't get in.

Registry Rules:

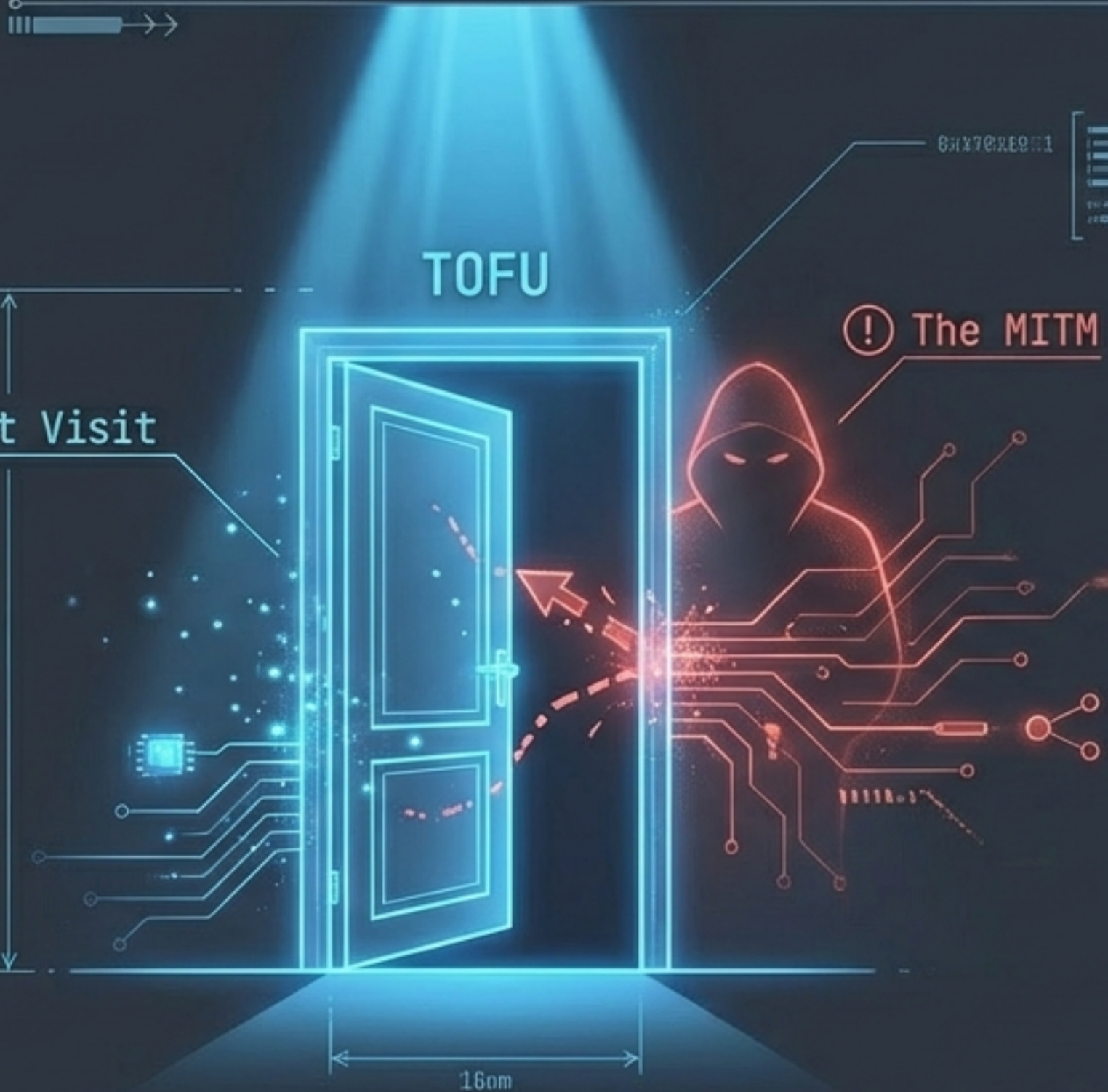
- [] Check Source Origin
- [] Check Content Type
- [] Check Signature Key

Result: Total isolation
from compromised CDNs or
injected scripts.

The Full Stack: Defense in Depth



Feature	TLS/HTTPS	HSTS	SRI	Service Worker Anchor
Protects Transport?	Yes	Yes	No	Yes
Protects Content?	No	No	No	Yes (Signed)
Persists Offline?	No	Yes	No	Yes
Restricts Sources?	No	No	No	Yes



- Trust On First Use (TOFU) is the Achilles heel.
- If the very first connection is intercepted, the attacker's keys are pinned, and the game is lost.
- Like SSH or Signal, we need an out-of-band verification method to secure the initial handshake.

Bridging the Gap: Physical Trust Anchors

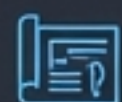


Mitigation Strategies

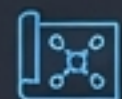
-  1. **The Business Card as Root of Trust:** Physical distribution of the trust bundle hash. 
-  2. **Multi-path Verification:** SW fetches trust bundle from DNS, GitHub, and Main Server simultaneously.
-  3. **Enterprise Pre-install:** MDM pushes the trust bundle before the browser opens.

Summary			
Project	Trust Bundle	Status	Complete
Version	1.0	Release Date	2023-10-27
Author	John Doe	Reviewer	Jane Smith
Approved		Signature	

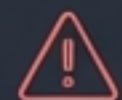
Trust Lifecycle and Key Rotation



The Key Continuity Chain: Each new trust bundle must be cryptographically signed by the previous key.



User Sovereignty: No silent updates. Critical key rotations require user interaction/confirmation.



Broken Chain = Critical Alert.

Implementation Roadmap

MVP: Code Pinning
(Cache & Hash bootstrap.js)

✓ Ready

Content Signing
(Ed25519 & Verification)

✓ Ready

Trust Registry
(Source Restriction)

⚙ In Design

Mitigations
(QR Codes, DNS TXT)

✂ In Design

Native Standard
(Browser Integration)

🔍 Research

0110100110010011000100100
11011110011010101011
01001110110101010101010101010101
111010101010101010101010101010101

The Bigger Picture: A New Web Standard



IETF



W3C



Firefox



Shield Browser

```
Pin-Script: sha256-abc123; max-age=31536000
```

```
Content-Signature: keyid="server-2026"; alg="ed25519"
```

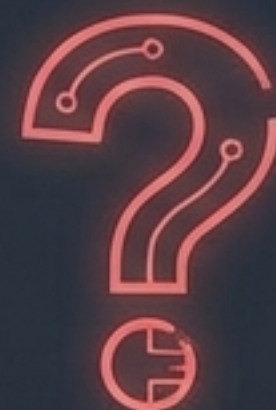
JetBrains Mono

Proposal: Move the logic from custom JavaScript (Service Worker) to Native Browser Headers.

Goal: Eliminate the JS layer entirely for performance and ubiquity.

Immediate Actions & Research Questions

- ✓ Implement Service Worker MVP (Phase 1)
- ✓ Simulate MITM attacks to verify detection
- ✓ Define Trust Bundle JSON schema



- Can a MITM force a Service Worker update?
- Can malicious extensions bypass the SW?
- Compliance: eIDAS/MiFID II data integrity?